INFORME DE EVALUACIÓN DE DESEMPEÑO

CC65 Programación Concurrente y Distribuida
Carrera de Ciencias de la Computación

Trabajo Final – Entregable 1 (PC3)
Caso 4: Predicción de Contrataciones y Riesgo en Compras Públicas

Sección: ……

Docente: Ms. Carlos Jara García

Alumnos (Grupo 5):
U_____ – Nombres y apellidos (integrante 1)
U_____ – Nombres y apellidos (integrante 2)
U_____ – Nombres y apellidos (integrante 3)

2026-20

CONTENIDO

1. Índice
2. Resumen
3. Descripción del problema y motivación
4. Objetivos
5. Desarrollo
6. Objetivo del estudiante
7. Demuestra ética profesional en el ejercicio de la ingeniería de software
8. Demuestra Responsabilidad profesional para el logro de los objetivos
9. Emite juicios considerando el impacto de las soluciones de ingeniería de software en el contexto global, impacto social, ambiental y económico
10. Conclusiones
11. Recomendaciones
12. Anexos
13. Bibliografía

1. ÍNDICE

2. RESUMEN....4
3. DESCRIPCIÓN DEL PROBLEMA Y MOTIVACIÓN....4
3.1. Contexto....4
3.2. Problema....4
3.3. Pregunta de impacto social....5
3.4. Motivación y beneficiarios....5
4. OBJETIVOS....5
4.1. Objetivo general....5
4.2. Objetivos específicos....5
5. DESARROLLO....6
5.1. Arquitectura de la solución....6
5.2. Limpieza y análisis de datos....6
5.3. Diseño del modelo ML distribuido....9
5.4. Paralelización del cálculo....10
5.5. Evidencias de la implementación y pruebas....13
5.6. Gestión del proyecto....15
6. OBJETIVO DEL ESTUDIANTE....15
7. Demuestra ética profesional en el ejercicio de la ingeniería de software....16
8. Demuestra Responsabilidad profesional para el logro de los objetivos....16
9. Emite juicios considerando el impacto de las soluciones de ingeniería de software en el contexto global, impacto social, ambiental y económico....16
10. CONCLUSIONES....17
11. RECOMENDACIONES....17
12. ANEXOS....17
13. BIBLIOGRAFÍA....18

2. RESUMEN

El Estado peruano realiza cada año cientos de miles de procedimientos de selección a través del Sistema Electrónico de Contrataciones del Estado (SEACE). Una parte importante de ellos no llega a buen término: se declaran desiertos, se cancelan o se anulan, lo que obliga a convocar de nuevo, retrasa obras y servicios públicos y consume recursos de las entidades. Con los datos abiertos del Organismo Especializado para las Contrataciones Públicas Eficientes (OECE), publicados en el estándar OCDS, se comprobó que el **20.6 %** de los procedimientos con resultado definido terminó de forma totalmente desfavorable (507 733 de 2 461 327).

Este primer entregable (PC3) construye, íntegramente en Go, las dos primeras piezas del sistema. La primera es un **cargador y limpiador concurrente** que procesa **5 145 770 ítems** y **2 760 591 procedimientos** en pocos segundos mediante un pipeline de goroutines y canales. La segunda es un **motor de entrenamiento paralelo** de regresión logística basado en paralelismo de datos (patrón *parameter server*), que alcanza una aceleración de **6.19x con 16 workers** sobre casi dos millones de filas sin alterar el resultado del modelo.

El modelo seleccionado, evaluado con validación temporal estricta (entrenamiento 2018–2023, validación 2024 y prueba 2025), detecta alrededor del **74 %** de los procedimientos que terminan desiertos o cancelados (AUC 0.663 en 2024 y 0.630 en 2025). El informe documenta además una lección metodológica clave: una partición aleatoria de los datos sobreestimaba el desempeño, por lo que se adoptó la validación temporal y se excluyeron variables con riesgo de fuga de información.

3. DESCRIPCIÓN DEL PROBLEMA Y MOTIVACIÓN

3.1. Contexto

Las contrataciones públicas son uno de los principales canales por los que el Estado convierte el presupuesto en obras, bienes y servicios para la ciudadanía. En el Perú, todos los procedimientos se registran en el SEACE y el OECE (antes OSCE) los publica como datos abiertos bajo el *Open Contracting Data Standard* (OCDS). El conjunto de datos utilizado abarca de 2003 a 2026 y contiene 2.76 millones de procedimientos y 5.15 millones de ítems convocados.

3.2. Problema

Un procedimiento puede terminar sin adjudicarse porque no se presentaron postores válidos (**desierto**), porque la entidad lo dejó sin efecto (**cancelado**) o porque se declaró su **nulidad**. En el análisis de los ítems se contabilizaron 840 576 ítems con detalle DESIERTO, 56 918 cancelados y más de 168 000 declarados nulos. Cada uno de estos casos implica repetir el proceso, extender los plazos y postergar la atención de una necesidad pública, como la construcción de una obra, la compra de medicamentos o la contratación de un servicio.

Hoy estas situaciones se detectan **después** de que ocurren. No existe una herramienta pública que, en el momento de la convocatoria, advierta qué procedimientos tienen mayor probabilidad de fracasar para que la entidad revise sus bases, plazos o requerimientos, o para que los órganos de control prioricen su supervisión.

3.3. Pregunta de impacto social

¿Qué procedimientos de contratación pública tienen alto riesgo de quedar desiertos o ser cancelados, de modo que las entidades y los órganos de control puedan intervenir antes de que se pierdan tiempo y recursos públicos?

3.4. Motivación y beneficiarios

- **Ciudadanía:** recibe a tiempo las obras y servicios financiados con sus impuestos cuando se reducen las convocatorias fallidas.
- **Entidades públicas:** pueden corregir a tiempo las bases de procedimientos con alto riesgo de quedar desiertos.
- **Órganos de control y sociedad civil:** pueden priorizar la revisión de procedimientos riesgosos con un criterio basado en datos.
- **Proveedores, en especial las MYPE:** se benefician de procesos mejor planificados, con más probabilidad de adjudicarse.

Desde la perspectiva del curso, el problema exige procesar millones de registros reales y entrenar un modelo de clasificación binaria, lo que justifica una solución concurrente y distribuida de alto rendimiento construida en Go.

4. OBJETIVOS

4.1. Objetivo general

Construir en Go una solución concurrente que cargue, limpie y procese más de un millón de registros de contrataciones públicas del Perú y entrene en paralelo un modelo de aprendizaje automático que estime el riesgo de que un procedimiento termine desierto o cancelado.

4.2. Objetivos específicos

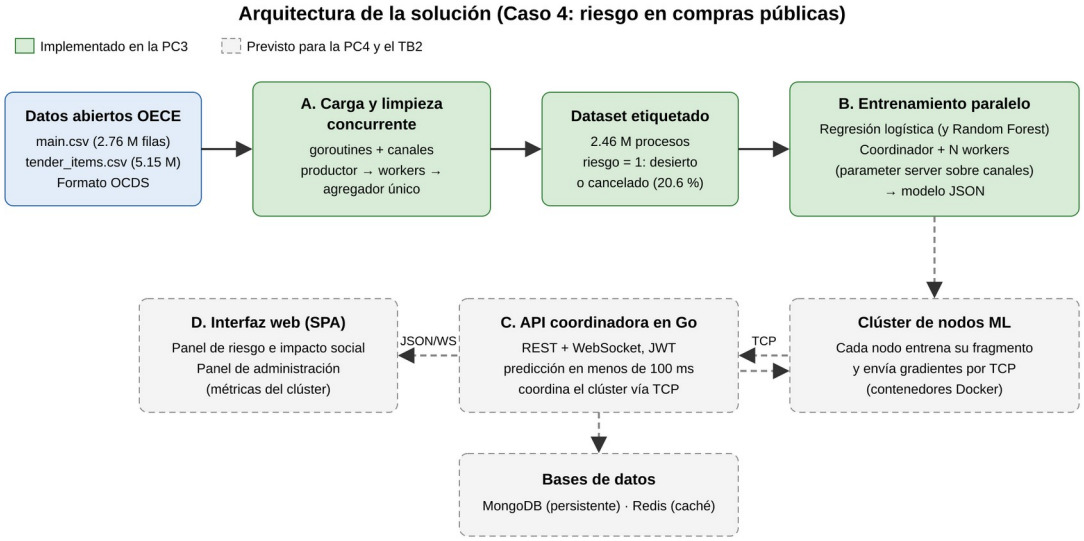
1. Cargar y limpiar de forma concurrente los archivos `main.csv` (2.76 M filas) y `tender_items.csv` (5.15 M filas) usando goroutines y canales.
2. Definir una etiqueta binaria de riesgo a partir del estado final de los ítems de cada procedimiento y analizar la calidad de los datos.
3. Diseñar una regresión logística distribuida basada en paralelismo de datos y justificar su elección frente a otros algoritmos.
4. Implementar el cálculo paralelo del gradiente garantizando la ausencia de condiciones de carrera y la reproducibilidad del resultado.
5. Evidenciar el funcionamiento con pruebas automatizadas, mediciones de aceleración y una evaluación honesta del modelo con validación temporal.

5. DESARROLLO

5.1. Arquitectura de la solución

La solución se organiza en las cuatro partes que propone el enunciado. En la PC3 se implementaron la parte A (carga y limpieza concurrente) y la parte B (entrenamiento paralelo); la API coordinadora, el clúster de nodos por TCP, las bases de datos y la interfaz web se construirán en la PC4 y el TB2 sobre el mismo diseño.

Figura 1. Arquitectura de la solución y alcance de la PC3.



Fuente: elaboración propia.

El flujo de la PC3 es lineal: los CSV del OECE se leen y limpian en paralelo, se genera un dataset etiquetado y, a partir de él, el motor de entrenamiento produce un modelo serializado en JSON que la API de la PC4 cargará para responder predicciones.

5.2. Limpieza y análisis de datos

a) Fuente de datos

Se utilizó la publicación del OECE en el Registro de Datos de la Open Contracting Partnership (descarga CSV de todos los años, corte del 4 de octubre de 2026, licencia CC BY 4.0). El conjunto se compone de varios archivos enlazados por la columna `main_ocid`; para este caso se usaron los dos siguientes.

Tabla 1. Archivos utilizados y tiempo de lectura completa en streaming (inspeccionar.go).

Archivo	Contenido	Filas	Columnas	Tamaño	Lectura
main.csv	Un procedimiento por fila	2 760 591	41	1.99 GB	1.97 s
tender_items.csv	Ítems de cada procedimiento	5 145 770	18	1.37 GB	1.53 s

Fuente: OECE (2026), procesado por el equipo.

b) Procedimiento de limpieza

Cada etapa es un programa en Go que lee los archivos en *streaming* (búfer de 1 MB y reutilización de registros), por lo que nunca carga los 3.4 GB completos en memoria.

Tabla 2. Etapas del procedimiento de limpieza y preparación.

Paso	Programa	Propósito	Resultado
1	inspeccionar.go	Contar filas y extraer encabezados	columnas_*.txt
2	analizar_estados.go	Distribución de status y statusDetails de los ítems	Tabla 3
3	analizar_procesos.go	Combinar los estados de los ítems de cada procedimiento	Tabla 4
4	limpiar_concurrente.go	Etiquetado concurrente con N workers	etiquetas_procesos.csv
5	preparar_dataset.go	Unir etiquetas con main.csv y depurar variables	entrenamiento_base.csv
6	preparar_dataset_controlado.go	Dataset sin variables con riesgo de fuga, años 2018–2025	entrenamiento_controlado.csv

Fuente: elaboración propia.

c) Análisis de estados

Tabla 3. Distribución de estados de los 5 145 770 ítems (analizar_estados.go, 1.93 s).

Estado (status)	Ítems	%	Detalle más frecuente (statusDetails)	Ítems
complete	3 660 141	71.1	BUENA PRO CONSENTIDA	2 297 699
unsuccessful	1 009 418	19.6	DESIERTO	840 576
(vacío)	387 153	7.5	CONTRATADO	771 404
cancelled	56 918	1.1	ADJUDICADO	510 433
active	32 140	0.6	NULO DE OFICIO / NULO	168 841

Fuente: elaboración propia.

Se observa que **uno de cada cinco ítems termina sin éxito**, principalmente por quedar desierto. Ningún ítem carece de `main_ocid`, lo que permite agruparlos por procedimiento sin pérdidas.

d) Regla de etiquetado

Como un procedimiento puede tener varios ítems con resultados distintos, cada estado se codifica como una bandera de bits y se combinan con un OR por procedimiento. La etiqueta `riesgo` se define así:

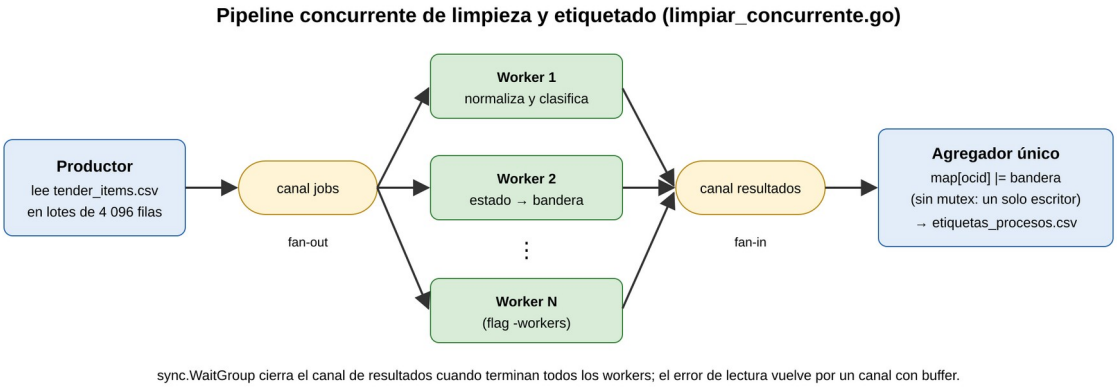
- `riesgo = 0` si **todos** sus ítems terminaron `complete`.
- `riesgo = 1` si **todos** sus ítems terminaron `unsuccessful` o `cancelled`.
- Se **excluyen** los procedimientos mixtos y los que tienen ítems activos, sin estado o con estados desconocidos, porque su resultado final es ambiguo.

Tabla 4. Resultado del etiquetado concurrente (limpiar_concurrente.go con 16 workers: 1.96 s).

Categoría	Procedimientos	%
Solo completos (riesgo = 0)	1 953 594	70.8
Solo desfavorables (riesgo = 1)	507 733	18.4
Mixtos (excluidos)	85 971	3.1
Indeterminados (excluidos)	213 099	7.7
Total de procedimientos únicos	**2 760 397**	**100.0**

Fuente: elaboración propia.

Figura 2. Pipeline concurrente de limpieza y etiquetado.



Fuente: elaboración propia.

e) Calidad de datos y decisiones de depuración

El análisis de calidad, apoyado en programas de auditoría del equipo (carpeta `diagnosticos/`), produjo las siguientes decisiones:

- **Montos:** la columna `tender_value_amount_PEN` solo está completa en el 34 % de los procedimientos, mientras que `tender_value_amount` lo está en el 100 % y el 97 % de los procedimientos está en soles. Se usa el monto original cuando la moneda es PEN, lo que recuperó **1 532 678 montos** y redujo los faltantes de 1.61 M a 78 554.
- **Tipo de procedimiento:** `tender_procurementMethod` solo distingue cinco valores, por lo que se incorporó `tender_procurementMethodDetails`, normalizado a mayúsculas y sin tildes para unificar variantes (por ejemplo, «Licitación Pública» y «LICITACION PUBLICA»).
- **Duración de la convocatoria:** se calculó en días a partir de las fechas de `tenderPeriod`, disponibles en el 68 % de los procedimientos.
- **Fuga de información:** se excluyó `tender_numberOfTenderers`, porque un procedimiento desierto es justamente uno sin ofertas válidas. Además, `main.csv` es un registro **consolidado** (*compiled release*) que puede contener valores actualizados después de la convocatoria; por ello se construyó un dataset **controlado** sin montos ni presupuesto y se eliminaron los textos que mencionan el resultado (por ejemplo, «DESIERTO» o «CANCELADO»).

Tabla 5. Datasets resultantes de la limpieza.

Dataset	Filas	Variables principales
entrenamiento_base.csv	2 461 327	monto (PEN), método, tipo de procedimiento, categoría, fecha, duración de la convocatoria
entrenamiento_controlado.csv	540 749 (2018–2025)	método, tipo de procedimiento, categoría, entidad compradora, mes, palabras del título y de la descripción

Fuente: elaboración propia.

5.3. Diseño del modelo ML distribuido

a) Elección del algoritmo

El caso 4 es un problema de **clasificación binaria** (riesgo sí/no), para el cual el enunciado recomienda Random Forest o regresión logística distribuida. Se eligió la **regresión logística distribuida** como modelo principal por tres razones: produce una probabilidad interpretable (cada peso indica cuánto aumenta o disminuye el riesgo una característica), su gradiente es una **suma independiente por fila** que se paraleliza de forma natural, y su estado completo es un vector de pesos pequeño, fácil de transmitir entre nodos por TCP en la PC4. Como comparación no lineal, el equipo implementó también un Random Forest en Go puro.

b) Formulación

Para un procedimiento con vector de características *x*, el modelo estima la probabilidad de riesgo como:

$$p(\text{riesgo} = 1 \mid x) = \sigma(w \cdot x + b), \quad \text{donde } \sigma(z) = 1 / (1 + e^{-z})$$

Los pesos se ajustan minimizando la entropía cruzada ponderada con regularización L2:

$$L(w) = -(1 / \Sigma c_i) \cdot \Sigma\, c_i \left[y_i \log p_i + (1 - y_i) \log(1 - p_i) \right] + (\lambda/2)\|w\|^2$$

donde c_i es el peso de la clase de la fila *i* (n / 2n_clase), que compensa el desbalance entre procedimientos exitosos (79 %) y fallidos (21 %). El gradiente es:

$$\nabla L(w) = (1 / \Sigma c_i) \cdot \Sigma\, c_i (p_i - y_i)\, x_i + \lambda w$$

c) Distribución del cálculo

Como el gradiente es una suma sobre las filas, puede descomponerse en sumas parciales: si los datos se dividen en *K* fragmentos (*shards*), cada worker calcula la suma de su fragmento y el coordinador suma los *K* resultados. El gradiente obtenido es **matemáticamente idéntico** al secuencial, por lo que paralelizar no cambia el modelo, solo el tiempo. Este es el esquema de **paralelismo de datos con servidor de parámetros** (Li et al., 2014): el coordinador mantiene los pesos y los workers solo calculan gradientes.

- **Optimizador del motor determinista (paquete ml):** descenso de gradiente con momento sobre el lote completo o sobre mini-lotes.

- **Optimizador del modelo controlado:** mini-lotes de 8 192 filas con Adam (Kingma y Ba, 2015) y parada temprana por AUC de validación.
- **Codificación:** variables numéricas normalizadas (logaritmo del monto y de la duración), codificación *one-hot* de categorías y *feature hashing* (Weinberger et al., 2009) para la entidad compradora y las palabras del título y de la descripción, con 13 037 dimensiones en total.
- **Validación temporal:** entrenamiento con 2018–2023, elección de umbral y época con 2024 y prueba exploratoria con 2025, para simular el uso real del modelo sobre procedimientos futuros.

En la PC4 este diseño se distribuye sin cambios conceptuales: cada nodo del clúster (contenedor Docker) mantendrá un fragmento del dataset y enviará sus gradientes parciales al coordinador por TCP, que es exactamente el papel que hoy cumplen las goroutines y los canales.

5.4. Paralelización del cálculo

a) Justificación del planteamiento

En una época de entrenamiento el costo dominante es calcular el gradiente: O(n · d) operaciones para *n* filas y *d* características (casi 2 millones × 70 en el dataset base). La reducción de los resultados parciales, en cambio, cuesta O(K · d) para *K* workers, es decir, unas pocas operaciones. Esta asimetría hace que el problema sea **de datos paralelos** casi perfecto: la fracción secuencial es mínima y la aceleración depende sobre todo del ancho de banda de memoria y del costo de sincronizar cada paso.

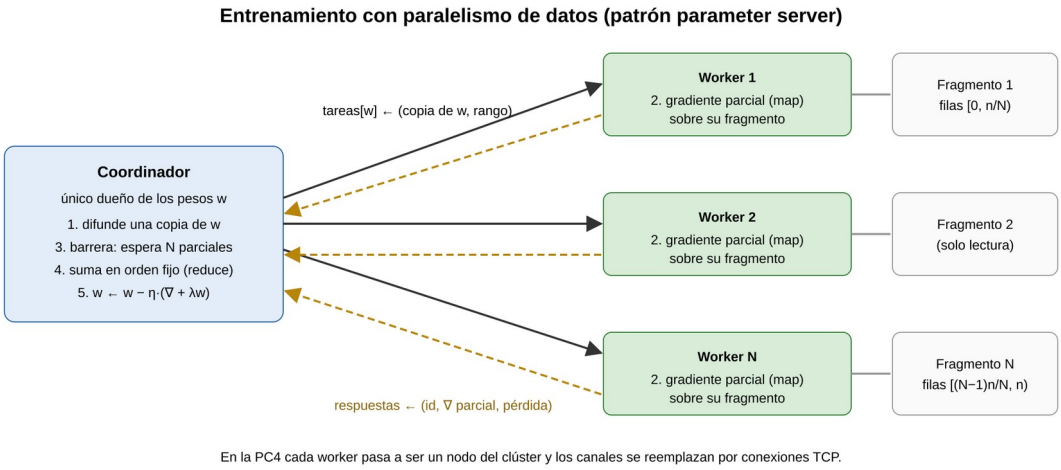
b) Técnicas y patrones de Go utilizados

Tabla 6. Patrones de concurrencia aplicados y su propósito.

Patrón / técnica	Dónde se aplica	Para qué sirve
Pipeline productor–consumidor	limpiar_concurrente.go	Separar la lectura del CSV del procesamiento
Worker pool + fan-out / fan-in	Limpieza y entrenamiento	Repartir lotes entre N goroutines y recoger sus resultados
Procesamiento por lotes (4 096 filas)	Limpieza	Reducir el costo de comunicación por canal
Confinamiento (agregador único)	Mapa de estados por procedimiento	Evitar mutex: solo una goroutine escribe el mapa
sync.WaitGroup	Limpieza, codificación y entrenamiento	Esperar a que terminen los workers y cerrar canales
Sharding de datos	Entrenamiento	Cada worker es dueño de un rango de filas
Barrera por paso	Coordinador del entrenamiento	Actualizar los pesos solo cuando llegaron todos los gradientes
Copia inmutable de los pesos	Coordinador → workers	Los workers leen una instantánea que nadie modifica
Reducción en orden fijo	Coordinador	Resultado reproducible aunque las goroutines terminen en otro orden
Escrituras en rangos disjuntos	Codificación paralela (ConstruirDatos)	Llenar una matriz compartida sin sincronización
sync.Once	Codificador	Construir índices una sola vez, seguro entre goroutines

Fuente: elaboración propia.

Figura 3. Entrenamiento con paralelismo de datos (patrón parameter server).



Fuente: elaboración propia.

c) Implementación del paso de entrenamiento

El siguiente extracto de `ml/entrenamiento.go` muestra el núcleo del patrón: cada worker es una goroutine con su propio canal de tareas, y el coordinador difunde la instantánea de pesos, espera todas las respuestas y reduce en orden fijo.

```
for w := range workers {                    // pool de workers persistente
    tareas[w] = make(chan tarea, 1)
    wg.Add(1)
    go func() {
        defer wg.Done()
        for t := range tareas[w] {          // map: gradiente del fragmento
            parcial := CalcularParcial(t.modelo, d, t.desde, t.hasta, pesos)
            respuestas <- respuesta{worker: w, parcial: parcial}
        }
    }()
}
...
instantanea := modelo.Clonar()              // copia inmutable de los pesos
for w := range workers {                    // difusión (broadcast)
    tareas[w] <- tarea{modelo: instantanea, desde: d0, hasta: d1}
}
for range activos {                         // barrera
    r := <-respuestas
    parciales[r.worker] = r.parcial
}
for w := range parciales {                  // reduce en orden fijo
    sumar(gradiente, parciales[w].Gradiente)
}
actualizarPesos(modelo, velocidad, gradiente, pesoPaso, cfg)
```

d) Ausencia de condiciones de carrera

Según el modelo de memoria de Go, el envío por un canal ocurre antes (*happens-before*) que la recepción correspondiente (The Go Authors, 2022). El diseño aprovecha esta garantía en todos los puntos de intercambio:

- Solo el coordinador escribe los pesos. Los workers reciben por el canal una copia que nadie modifica después del envío.
- Los datos de entrenamiento son de solo lectura una vez construidos.
- Cada worker escribe únicamente su propio gradiente parcial, y el coordinador lo lee después de recibirlo por el canal.
- En la limpieza, el mapa de procedimientos lo modifica una sola goroutine (el agregador), por lo que no requiere mutex.

Estas propiedades se verificaron ejecutando todas las pruebas con el detector de carreras de Go (`go test -race`), sin advertencias.

5.5. Evidencias de la implementación y pruebas

a) Organización del código

Tabla 7. Estructura del repositorio.

Ruta	Responsabilidad
inspeccionar.go, analizar_*.go	Exploración y análisis de estados
limpiar_concurrente.go	Limpieza y etiquetado concurrente (pipeline con workers)
preparar_dataset.go, preparacion/	Construcción de los datasets de entrenamiento
ml/	Paquete del modelo: carga, codificación paralela, entrenamiento paralelo, métricas y persistencia
cmd/entrenar/	Comando de entrenamiento, evaluación y comparación de workers
experimentos/	Modelos controlados con validación temporal y Random Forest
diagnosticos/	Auditorías de calidad, fuga de información y temporalidad

Fuente: elaboración propia.

b) Ejecución de la limpieza concurrente

```
$ go run limpiar_concurrente.go -workers 16
=== LIMPIEZA CONCURRENTE ===
Workers utilizados: 16
Items procesados: 5145770
Items sin OCID: 0
Procedimientos unicos: 2760397
Clase 0: 1953594
Clase 1: 507733
Mixtos excluidos: 85971
Indeterminados excluidos: 213099
Archivo generado: data/etiquetas_procesos.csv
Tiempo total: 1.963826551s
```

c) Pruebas automatizadas

El paquete `ml/` cuenta con 11 pruebas y un benchmark escritos con la biblioteca estándar `testing`. Todas pasan con el detector de carreras activado:

```
$ go test -race ./ml
ok      grupo5-prograconcurrente/ml     2.032s
```

Tabla 8. Pruebas automatizadas del paquete ml.

Prueba	Qué verifica
TestLeerRegistros (6 casos)	Lectura y validación del CSV: columnas faltantes, etiquetas inválidas, montos y fechas
TestCodificador	Normalización, one-hot, valores no vistos y frecuencia mínima
TestConstruirDatosParaleloIgualSecuencial	La codificación con 2, 7 y 16 goroutines produce la misma matriz que con 1
TestCalcularParcialCoincideConGradienteNumerico	El gradiente analítico coincide con la derivada numérica
TestEntrenarParaleloIgualSecuencialConLoteCompleto	Con 2, 3 y 8 workers los pesos son iguales a los de 1 worker (tolerancia 1e-9)
TestEntrenarParaleloEsReproducible	Dos ejecuciones con la misma configuración dan exactamente los mismos pesos
TestEntrenarParaleloAprendeLaRegla	El modelo recupera una regla de riesgo conocida (AUC > 0.75)
TestEntrenarParaleloRechazaConfigInvalida (7 casos)	Validación de hiperparámetros
TestEntrenarConMasWorkersQueFilas	Funciona cuando hay más workers que filas
TestEvaluar	Matriz de confusión, precisión, sensibilidad, F1 y AUC calculados a mano
TestArchivoModeloIdaYVuelta	Guardar y cargar el modelo produce las mismas predicciones

Fuente: elaboración propia.

Para comprobar que las pruebas detectan errores reales, se invirtió deliberadamente el signo del gradiente del sesgo: la prueba del gradiente numérico falló como se esperaba y volvió a pasar al restaurar el código.

d) Rendimiento del entrenamiento paralelo

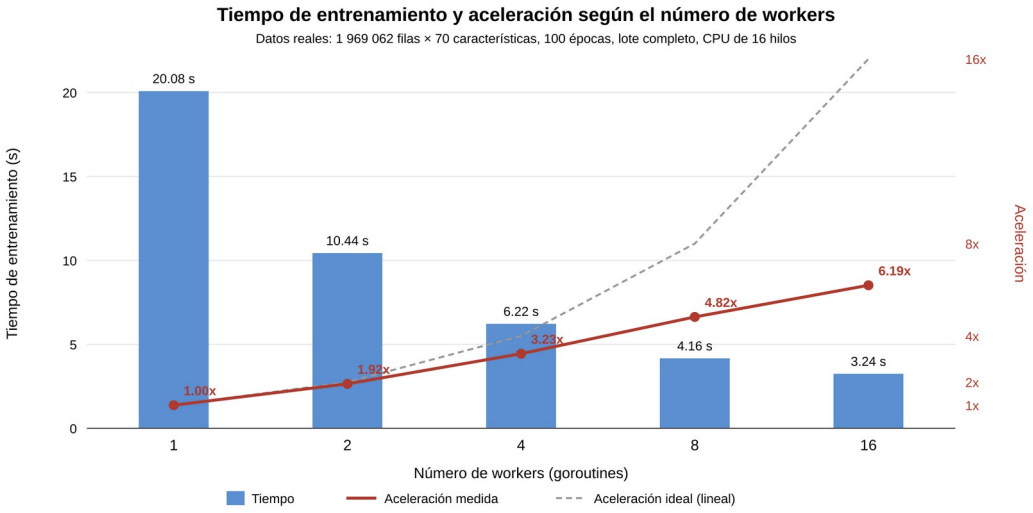
Se entrenó el mismo modelo variando solo el número de workers (`go run ./cmd/entrenar -comparar 1,2,4,8,16`), con 1 969 062 filas de entrenamiento, 70 características y 100 épocas de lote completo, en un equipo con 16 hilos de CPU.

Tabla 9. Aceleración del entrenamiento según el número de workers (datos reales).

Workers	Tiempo	Aceleración	Eficiencia	Pérdida final	AUC
1	20.08 s	1.00x	100 %	0.660293	0.6365
2	10.44 s	1.92x	96 %	0.660293	0.6365
4	6.22 s	3.23x	81 %	0.660293	0.6365
8	4.16 s	4.82x	60 %	0.660293	0.6365
16	3.24 s	6.19x	39 %	0.660293	0.6365

Fuente: elaboración propia.

Figura 4. Tiempo de entrenamiento y aceleración según el número de workers.



Fuente: elaboración propia.

La pérdida y el AUC son **idénticos** con cualquier número de workers, lo que confirma que la paralelización no altera el resultado. La eficiencia es casi ideal con 2 workers (96 %) y desciende con más workers porque el cálculo por fila es muy liviano: el ancho de banda de memoria y la barrera de sincronización de cada paso pasan a ser el límite, como predice la ley de Amdahl.

e) Resultados del modelo

Tabla 10. Comparación de modelos sobre datos reales (umbral elegido en validación).

Modelo	Validación	AUC	AP	Precisión	Sensibilidad	F1
Regresión logística controlada	2024	0.663	0.307	0.269	0.734	0.393
Regresión logística controlada	2025	0.630	0.302	0.276	0.741	0.402
Random Forest controlado (60 árboles)	2024	0.653	0.296	0.248	0.916	0.391
Regresión logística base	Aleatoria 20 %	0.637	–	0.258	0.736	0.382
Regresión logística base	2025	0.545	–	0.254	0.193	0.220
Referencia: predecir siempre riesgo	2024	0.500	0.202	0.202	1.000	0.336

Fuente: elaboración propia.

El **modelo seleccionado es la regresión logística controlada**: con validación temporal detecta el 74 % de los procedimientos que terminan desiertos o cancelados y mantiene su desempeño en 2025 (AUC 0.630), con una precisión alrededor de un tercio mayor que la tasa base de procedimientos fallidos (0.276 frente a 0.202). El modelo base obtenía un AUC similar con una partición aleatoria, pero **su desempeño cae a 0.545 en 2025**, porque dependía del año y de los montos, cuya distribución cambia con el tiempo. Este contraste justifica la validación temporal y la exclusión de variables con riesgo de fuga.

El entrenamiento del modelo controlado con 16 workers toma 0.59 s y el del Random Forest 1.11 s, por lo que ambos modelos pueden reentrenarse con frecuencia cuando se publiquen nuevos datos.

5.6. Gestión del proyecto

El código se aloja en un repositorio público de GitHub (Anexo A) y sigue Git Flow: el trabajo se desarrolla en ramas `feature/*` (por ejemplo, `feature/entrenamiento-paralelo` y `feature/regresion-logistica-paralela`), se integra en `develop` mediante merges sin *fast-forward* y, una vez verificado, se publica en `main`. El seguimiento de tareas se realiza en el tablero Kanban del proyecto en GitHub (Anexo C).

6. OBJETIVO DEL ESTUDIANTE

ABET – EAC - Student Outcome 4: La capacidad de reconocer responsabilidades éticas y profesionales en situaciones de ingeniería y hacer juicios informados, que deben considerar el impacto de las soluciones de ingeniería en contextos globales, económicos, ambientales y sociales.

El trabajo aporta a esta competencia porque obliga a tomar decisiones de ingeniería cuyas consecuencias trascienden el código: un modelo que señala procedimientos de contratación como riesgosos puede influir en cómo se asignan recursos de control y en la reputación de entidades públicas. Por ello, en cada etapa se priorizó la transparencia de los datos, la honestidad en la evaluación y la utilidad social del resultado, como se detalla a continuación.

7. Demuestra ética profesional en el ejercicio de la ingeniería de software

- **Uso responsable de datos abiertos:** se utilizaron exclusivamente datos públicos del OECE bajo licencia CC BY 4.0, citando su fuente, y no se procesaron datos personales.
- **Honestidad en los resultados:** al detectar que una partición aleatoria inflaba el desempeño (AUC 0.637 frente a 0.545 en 2025), se reportó el resultado menos favorable y se adoptó la validación temporal, en lugar de presentar la cifra más alta.
- **Prevención de fuga de información:** se excluyeron el número de postores, los montos del registro consolidado y los textos que revelan el resultado, aunque mejoraran artificialmente las métricas.
- **Uso del modelo como apoyo y no como acusación:** la probabilidad de riesgo orienta la revisión preventiva de un procedimiento; no implica irregularidades de la entidad ni reemplaza el juicio de los especialistas.

8. Demuestra Responsabilidad profesional para el logro de los objetivos

- Se cumplieron los requisitos de la PC3: carga y procesamiento concurrente de más de un millón de registros (5.15 M ítems), diseño del modelo distribuido, paralelización del cálculo y evidencias de implementación.
- La calidad se respaldó con 11 pruebas automatizadas ejecutadas con el detector de carreras, un benchmark y mediciones de aceleración reproducibles.
- El trabajo se organizó con Git Flow, ramas por funcionalidad y un tablero Kanban, integrando las contribuciones del equipo sin conflictos.
- El diseño se preparó para la PC4: el motor de entrenamiento ya separa coordinador y workers, por lo que la distribución en nodos TCP no requiere rediseñar el algoritmo.

9. Emite juicios considerando el impacto de las soluciones de ingeniería de software en el contexto global, impacto social, ambiental y económico

- **Impacto social:** anticipar procedimientos con riesgo de quedar desiertos permite que obras y servicios públicos, como infraestructura educativa o sanitaria, lleguen a tiempo a la población.
- **Impacto económico:** cada convocatoria fallida implica repetir trámites, extender plazos y, en obras, encarecer los costos; reducirlas libera recursos del Estado y de los proveedores, especialmente de las MYPE.
- **Contexto global:** al basarse en el estándar internacional OCDS, la solución puede adaptarse a datos de otros países que publican sus contrataciones con el mismo estándar.
- **Impacto ambiental:** el uso eficiente de los núcleos disponibles permite entrenar con casi dos millones de registros en segundos en un equipo personal, sin requerir infraestructura de alto consumo energético.

10. CONCLUSIONES

1. Se construyó en Go un pipeline concurrente que procesa los 5 145 770 ítems y los 2 760 591 procedimientos del OECE en alrededor de dos segundos por etapa, cumpliendo el requisito de procesar más de un millón de registros.
2. El 20.6 % de los procedimientos con resultado definido terminó desierto o cancelado, lo que confirma la relevancia del problema y la utilidad de anticiparlo.
3. La regresión logística con paralelismo de datos alcanzó una aceleración de 6.19x con 16 workers, con resultados idénticos a la ejecución secuencial y sin condiciones de carrera, verificado con `go test -race`.
4. El modelo seleccionado detecta el 74 % de los procedimientos fallidos con validación temporal (AUC 0.663 en 2024 y 0.630 en 2025); es útil para priorizar revisiones, aunque su precisión todavía es moderada.
5. La validación temporal y el control de la fuga de información fueron decisivos: una evaluación aleatoria habría sobreestimado el desempeño real del modelo sobre procedimientos futuros.

11. RECOMENDACIONES

1. Distribuir en la PC4 el motor de entrenamiento en nodos Docker que intercambien gradientes por TCP, reutilizando el diseño de coordinador y workers.
2. Incorporar el historial de cada entidad compradora (tasa de procedimientos fallidos en años anteriores), calculado solo con información del pasado, para mejorar la precisión.
3. Obtener las publicaciones históricas (*releases*) del OECE para verificar qué información estaba disponible al momento de la convocatoria y ampliar el conjunto de variables con seguridad.
4. Unificar los experimentos en el paquete `ml/` para que el modelo seleccionado se beneficie del motor determinista y de sus pruebas.
5. Calibrar el umbral de decisión según la capacidad de revisión de los usuarios finales, priorizando la precisión cuando los recursos de control sean limitados.

12. ANEXOS

Anexo A. Repositorio del proyecto

Repositorio GitHub: https://github.com/YuhiTTo/GRUPO5-PrograConcurrente

Ramas: `main` (entrega), `develop` (integración), `feature/entrenamiento-paralelo` y `feature/regresion-logistica-paralela`.

Anexo B. Reproducción de los resultados

```
# Descargar el CSV de todos los años del OECE y descomprimirlo en data/
go run inspeccionar.go
go run analizar_estados.go
go run analizar_procesos.go
go run limpiar_concurrente.go -workers 16
go run preparar_dataset.go
go run ./cmd/entrenar
go run ./cmd/entrenar -comparar 1,2,4,8,16
go run ./preparacion/preparar_dataset_controlado.go
go run ./experimentos/entrenar_paralelo_controlado.go \
    -desde 2018 -validacion 2024 -workers 16 \
    -usar-descripcion=true -evaluar-prueba=true
go test -race ./ml
```

Anexo C. Reporte de participación y tablero Kanban

Tabla 11. Reporte de participación según el historial de commits.

Integrante	Usuario GitHub	Contribución principal	%
(completar)	Littlepie4, YuhiTTo	Creación del repositorio; inspección, análisis de estados y limpieza concurrente	(completar)
(completar)	SeuNg720p	Entrenamiento paralelo con validación temporal, modelos controlados, Random Forest y auditorías de datos	(completar)
(completar)	Jaed69	Paquete ml (motor paralelo determinista), pruebas y benchmark, mejora de variables e integración Git Flow	(completar)

Fuente: historial de Git del repositorio. Completar nombres, códigos y porcentajes acordados por el equipo.

Insertar aquí la captura del tablero Kanban del proyecto en GitHub.

Anexo D. Salida del entrenamiento del modelo controlado

```
Train (2018-2023): 393181 | Validacion (2024): 73145 | Prueba: 74423
Carga: 3.126343382s | Dimensiones: 13037 | Descripcion activada: true
Train clase 0=303700 clase 1=89481 | Peso positivo=1.00
Epoca 1/80 | loss val 0.47681 | AUC val 0.66056 | mejor AUC 0.66056
Parada temprana: 12 epocas sin mejora de AUC validacion
=== MODELO CONTROLADO SIN MONTO: RESULTADOS VALIDACION ===
Mejor epoca: 2 | Entrenamiento total: 590.03511ms
AUC: 0.6627 | Average Precision: 0.3065
Umbral (validacion): 0.22 | F1 val: 0.3934 | Precision: 0.2686 | Recall: 0.7344
EXPLORATORIO (2025): AUC 0.6295 AP 0.3018 F1 0.4016
                     Precision 0.2755 Recall 0.7410
```

13. BIBLIOGRAFÍA

Breiman, L. (2001). Random forests. *Machine Learning, 45*(1), 5–32. https://doi.org/10.1023/A:1010933404324

Cox-Buday, K. (2017). *Concurrency in Go: Tools and techniques for developers*. O'Reilly Media.

Donovan, A. A. A., & Kernighan, B. W. (2015). *The Go programming language*. Addison-Wesley.

Fawcett, T. (2006). An introduction to ROC analysis. *Pattern Recognition Letters, 27*(8), 861–874. https://doi.org/10.1016/j.patrec.2005.10.010

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2.ª ed.). Springer.

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. En *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*. https://arxiv.org/abs/1412.6980

Li, M., Andersen, D. G., Park, J. W., Smola, A. J., Ahmed, A., Josifovski, V., Long, J., Shekita, E. J., & Su, B.-Y. (2014). Scaling distributed machine learning with the parameter server. En *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)* (pp. 583–598). USENIX Association.

Open Contracting Partnership. (2026). *Perú: Organismo Especializado para las Contrataciones Públicas Eficientes (OECE)* [Conjunto de datos]. Registro de Datos OCP. https://data.open-contracting.org/es/publication/135

Open Contracting Partnership. (s. f.). *Open Contracting Data Standard: Documentation*. https://standard.open-contracting.org/

The Go Authors. (2022). *The Go memory model*. https://go.dev/ref/mem

Weinberger, K., Dasgupta, A., Langford, J., Smola, A., & Attenberg, J. (2009). Feature hashing for large scale multitask learning. En *Proceedings of the 26th International Conference on Machine Learning* (pp. 1113–1120). https://doi.org/10.1145/1553374.1553516